

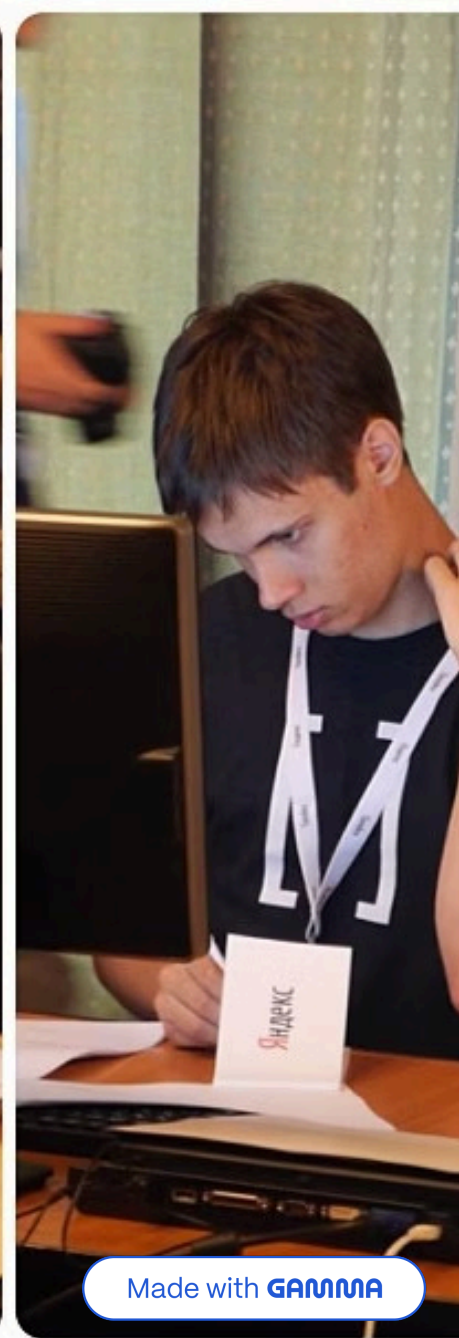
Week 3: Running the Machine

Auto pipe runs · Hidden test cases · Score-driven iteration

 by Alwin Lin

Welcome & Intro to Competitive Programing

- What is Competitive Programing?
- Where did Machine Learning in MLE-Star go?
- Why are we doing this?
- What does it look like?



The problem with pass/fail

- What does it do right now?
- What knowledge do we need to pass onto the LLM?
- What would be it be looking for?

G. Drowning

time limit per test: 2 seconds

memory limit per test: 256 megabytes

an array a consisting of n positive integers.

a reduction operation on any array c of length $|c| \geq 3$:

an index i ($1 < i < |c|$) such that $c_{i-1} + c_{i+1} > c_i$.

the triplet $\{c_{i-1}, c_i, c_{i+1}\}$ with a single integer $x = c_{i-1} - c_i + c_{i+1}$.

integer x occupies the position previously held by the triplet, and the length of the array decreases by 2.

is considered *good* if it can be reduced to a single element by performing the operation above zero or more times. Note that an array of length 1 is always good.

you are to count the number of pairs (l, r) ($1 \leq l \leq r \leq n$) such that the subarray $a[l, r]$ is good.

The input contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of each test case follows.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the size of the array.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

For each test case, output a single integer — the number of good subarrays.

```
1:
distance(5, [[1, 3], [4, 2], [3, 3]])
```

```
own:
] clockwise = 2 segments (1→2→3)
] clockwise = 3 segments (4→5→1→2)
] same stop = 0 segments
2 + 3 + 0 = 5
```

```
2:
distance(4, [[4, 1], [1, 4]])
```

```
own:
] clockwise = 1 segment (wraps)
] clockwise = 3 segments (1→2→3→4)
1 + 3 = 4
```

```
"hidden_cases": [
  # (args_dict, expected_output)
  ({ "n": 5, "trips": [[1,3],[4,2],[3,3]]}, 5), # given ex
  ({ "n": 4, "trips": [[4,1],[1,4]]}, 4), # given e
  ({ "n": 4, "trips": [[1,4]]}, 3), # near-fu
  ({ "n": 6, "trips": [[2,2],[3,3]]}, 0), # all sam
  ({ "n": 10, "trips": [[1,10],[10,1]]}, 10), # 9 + 1
  ({ "n": 3, "trips": [[1,2],[2,3],[3,1]]}, 3), # sequent
  ({ "n": 5, "trips": []}, 0), # empty t
  ({ "n": 1, "trips": [[1,1]]}, 0), # single
  ({ "n": 8, "trips": [[3,7],[7,3]]}, 8), # symmetr
  ({ "n": 6, "trips": [[5,1],[6,2],[1,6]]}, 9), # 2+2+5
]
```

Core concept: hidden test cases & the score

- What is seen:
 - Code description
 - Examples

Issues: An LLM Too Smart

- Test driven development works extremely well with LLM
- [vinext.js](https://vinext.js.com/)
 - \$ 1100 API cost
 - 700 Vitest tests
 - 380 Playwright end-to-end tests
 - 94% API coverage
 - 1 Weekend



Problem description + examples



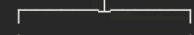
LLM generates code



solution.py



subprocess → run_cases.py



score = 1.0



Done

score < 1.0



Build failure message:

"Case 3: expected 5, got 7"

"Case 8: expected 0, got None"



Inject into next prompt



LLM retries

(loop back up ↑)



max_retries hit → halt

Closing the loop: score-driven iteration

- Start running the code
 - The failure message is the key
 - Only share the first 3 failures.
 - `max_retries` is mandatory
 - The score is the loop signal

Code Goes here: